



BUSINESS ANALYTICS

INDIVIDUAL CASE STUDY

Predicting Resource Bottlenecks and Cloud Cost Spikes in Distributed Job Processing Architectures

Submitted by

ADHIKKESH S K

CB.SC.U4CSE23101

B.Tech. Computer Science and Engineering

Department of Computer Science and Engineering

Amrita School of Computing, Coimbatore

Amrita Vishwa Vidyapeetham

September 2026

1. Problem Statement and Objectives

Data-driven organisations run web scraping, document processing, ETL and machine-learning jobs on *distributed job-processing architectures*: jobs are published to a message broker such as Apache Kafka and pulled by worker nodes that run on rented cloud virtual machines. Each node has a fixed CPU, memory and hourly price, whereas incoming jobs vary widely in size. When a heavy job is dispatched to a node that is already busy or short of free memory, the node becomes a **bottleneck**—the CPU saturates, memory is exhausted, the job runs several times longer, is retried or fails—and the cloud bill rises. Because the risk of a job is unknown before execution, teams either over-provision the whole cluster “just in case” or react after incidents. Production studies show that cloud workload behaviour is learnable [6], so the risk can be anticipated.

Business problem: *Can the bottleneck risk of an incoming job be predicted before it runs, using only information available at dispatch time, so that risky jobs are rerouted and cloud capacity is tied to actual risk?* The problem matters for **reliability** (bottleneck jobs here run a median 6.1 times longer), **cost** (slow jobs and blanket over-provisioning are avoidable spend) and **operations** (proactive, automated scheduling instead of fire-fighting).

Objectives:

1. **Diagnose the root cause** – identify the job attributes and node conditions that drive CPU and memory bottlenecks.
2. **Build an early-warning model** – classify an incoming job as *Bottleneck* (high risk) or *Normal* (low risk) from pre-execution data.
3. **Enable cost-aware scaling** – quantify the cost impact of bottlenecks and recommend scheduling and scaling actions.

2. Data Collection and Dataset Description

Company job logs are confidential and unpublished, so the data were collected in two stages (Figure 1): real infrastructure data were **web-scraped from a publicly accessible website**, and job-level telemetry was **generated synthetically on top of the scraped records**. No ready-made dataset from Kaggle, UCI, GitHub or similar repositories was used.

STAGE 1 - WEB SCRAPING (real data)

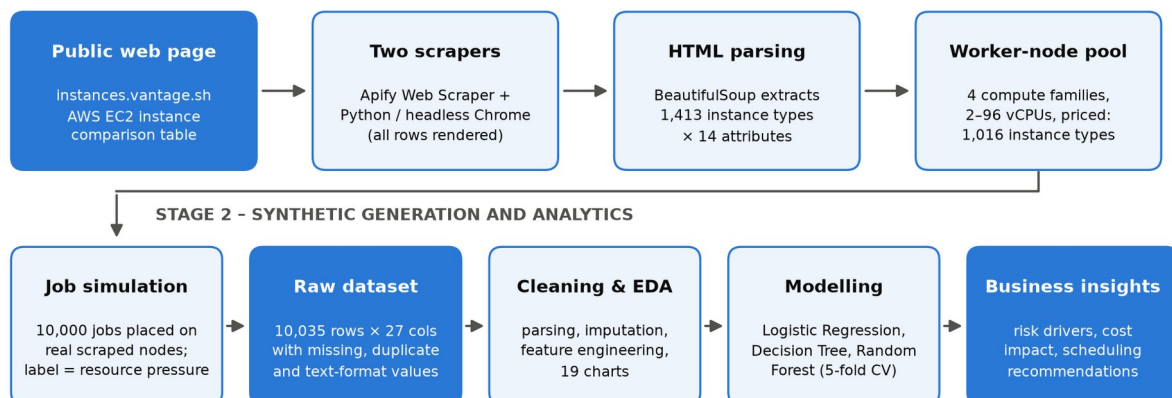


Figure 1. Two-stage data collection and analytics workflow.

Table 1. Sample of scraped records, exactly as displayed on the website.

API name	Compute family	Memory	vCPUs	Network	Linux on-demand	Linux spot (min.)
m5.large	General purpose	8 GiB	2 vCPUs	Up to 10 Gigabit	\$0.0960 hourly	\$0.0399 hourly
c6i.2xlarge	Compute optimized	16 GiB	8 vCPUs	Up to 12.5 Gigabit	\$0.3400 hourly	\$0.1413 hourly
r6g.4xlarge	Memory optimized	128 GiB	16 vCPUs	Up to 10 Gigabit	\$0.8064 hourly	\$0.2183 hourly
i4i.xlarge	Storage optimized	32 GiB	4 vCPUs	Up to 10 Gigabit	\$0.3430 hourly	\$0.1035 hourly
t3.medium	General purpose	4 GiB	2 vCPUs (4h 48m burst)	Up to 5 Gigabit	\$0.0416 hourly	\$0.0161 hourly

2.2 Stage 2 – Synthetic workload generation on the scraped records

The catalogue contains 449 memory-, 419 general-purpose, 329 compute- and 112 storage-optimised types plus 104 accelerator types. The generator (code in the notebook appendix) worked as follows:

- **Worker nodes:** 1,016 scraped types of the four general compute families (2–96 vCPUs, ≤ 768 GiB, with a Linux price) form the node pool; each job is placed on one of them (smaller nodes more often, probability $\propto 1/\sqrt{\text{vCPU}}$), so 979 real instance types appear in the data.
- **Jobs and node state:** 10,000 jobs over 90 days (1 June – 29 August 2026) with six workload categories, four priorities, four regions, log-normal payload sizes, and realistic distributions for CPU load, free pod memory, threads, broker latency, queue wait, I/O and garbage-collection (GC) pause at dispatch.
- **Label:** a resource-pressure score combines the job's memory demand relative to the **free memory of its scraped node**, CPU load, threads per vCPU, queue wait, latency, GC pause, retries, replicas, category and priority plus Gaussian noise; a logistic function turns it into the probability of *Bottleneck* (33.0% of jobs).
- **Outcomes and quality issues:** run time and job cost (= run hours $\times$ scraped hourly price $\times$ replicas); realistic defects were injected (inconsistent spellings, missing telemetry, impossible readings, a -1 sentinel, duplicate log writes).

2.3 Dataset description

The submitted file `cloud_job_execution_dataset.csv` has **10,035 records** (10,000 jobs + 35 duplicate writes) and **27 attributes**: 7 scraped node attributes (instance type, family, vCPUs, memory, network, on-demand and spot price), 17 simulated job and node-state attributes, 2 post-execution outcomes (execution_time_sec, job_cost_usd) and the target target_label (Bottleneck 3,304, 33.0%; Normal 6,696, 67.0%). Scraped node prices range from \$0.051 to \$13.97 per hour (Figure 3a); the target is moderately imbalanced (Figure 3b).

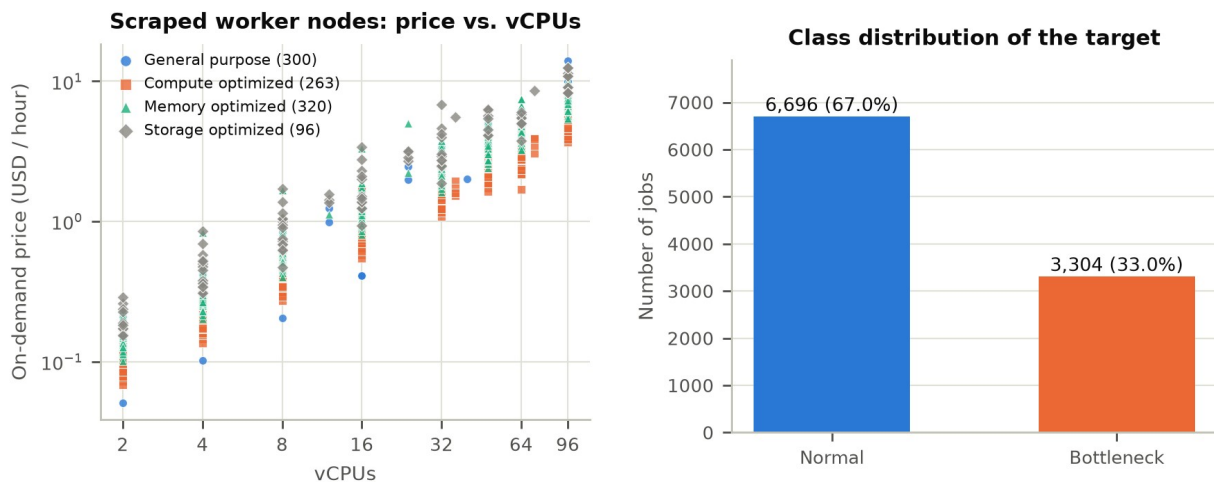


Figure 3. (a) Scraped instance types used as worker nodes – price rises almost proportionally with vCPUs; (b) class distribution of the target.

3. Data Preparation and Exploratory Analysis

3.1 Cleaning and preprocessing

Table 2. Data-quality problems and preprocessing steps.

Problem	Detection	Affected	Treatment
Duplicate log writes	Exact duplicate rows	35	Removed → 10,000 jobs
Inconsistent labels	“webscraping”, “Web Scraping”, “US-EAST”	500	Trimmed, lower-cased, mapped to canonical values
Numbers stored as text	“8 vCPUs”, “16 GiB”, “\$0.0960 hourly”	all rows	Regular-expression extraction; legacy “Moderate/High” network → 1/10 Gbps
Invalid readings	CPU > 100 % (22); latency = -1 (18)	40	Set to missing, then imputed
Missing telemetry	NaN in 6 columns (0.2–2.5 %)	820 cells	Median within the same job category
Skewed values	IQR rule	737 payloads	Kept (genuine heavy jobs); log-transformed

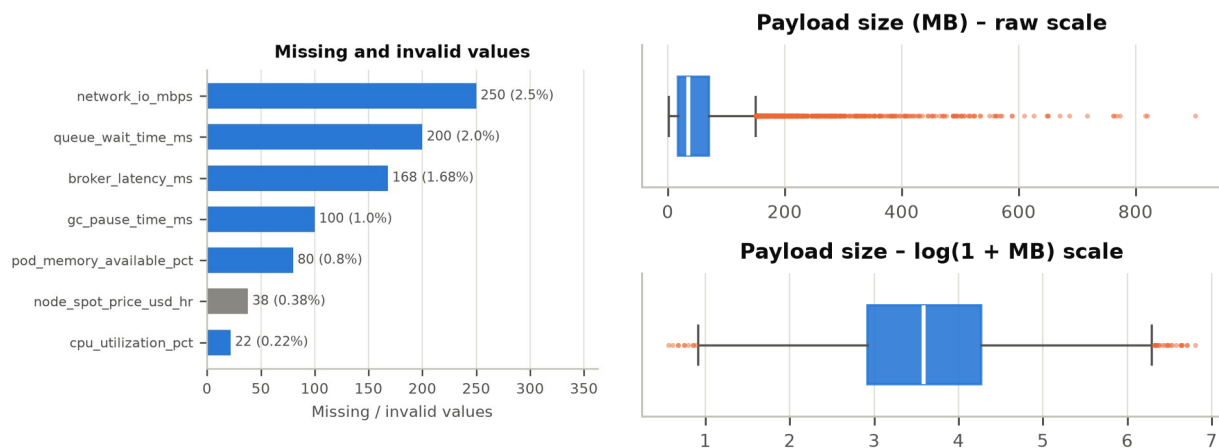


Figure 4. (a) Missing and invalid values before imputation (spot price is structurally missing); (b) payload size on raw and log scales.

Feature engineering: $\text{memory_pressure_ratio} = \text{payload} \div \text{free node memory (scraped memory} \times \text{available \%)}$, which measures job size *relative to the node it lands on*; threads_per_vcpu (CPU contention); $\text{memory_per_vcpu_gib}$; submit_hour and is_peak_hour (09:00–17:59); and the binary target is_bottleneck .

3.2 Exploratory data analysis

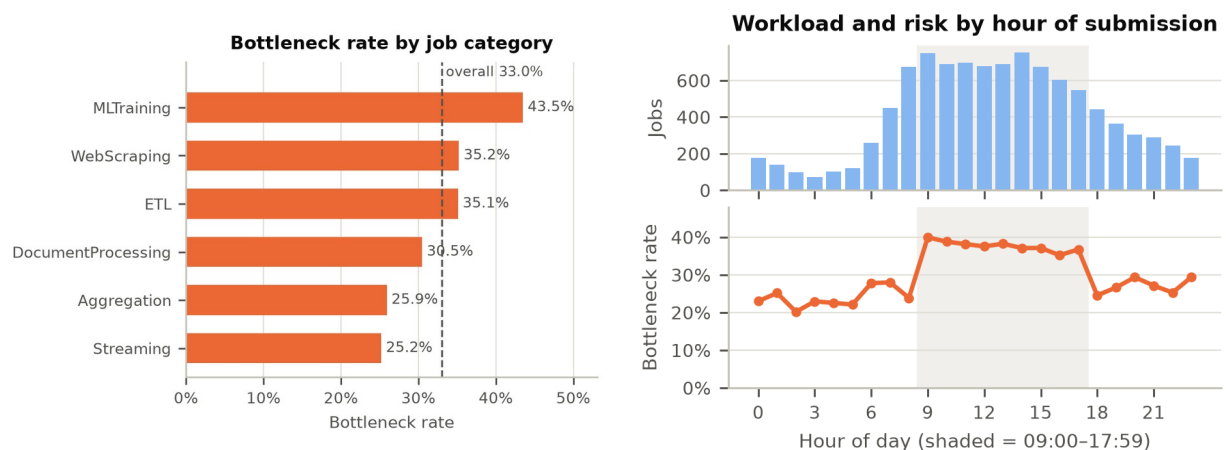


Figure 5. (a) Bottleneck rate by job category; (b) job volume and bottleneck rate by hour of submission.

Workload type and timing matter. ML-training jobs bottleneck most often (43.5%), followed by web scraping (35.2%) and ETL (35.1%); streaming (25.2%) and aggregation (25.9%) are safest (Figure 5). Risk also peaks with load during working hours (37.7% versus 25.8% off-peak), while *Critical* jobs bottleneck less than *Low* priority jobs (28.3% versus 35.3%).

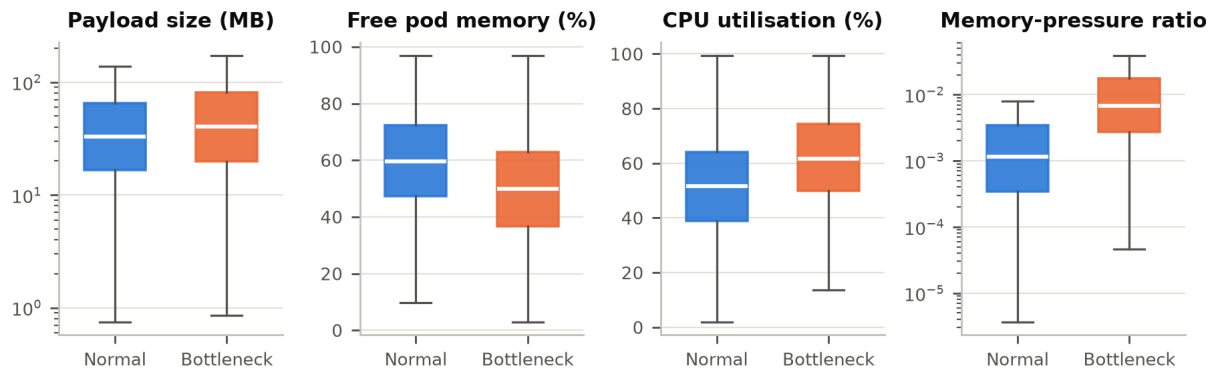


Figure 6. Key pre-execution variables for normal (blue) and bottleneck (orange) jobs.

Bottleneck jobs are dispatched with less free memory (median 49.9 % versus 59.5 %) and higher CPU load (61.8 % versus 51.7 %) (Figure 6). Payload alone differs only 1.23×, but the **memory-pressure ratio is 5.9× higher**, because bottleneck jobs land on smaller nodes (median 16 GiB versus 64 GiB).

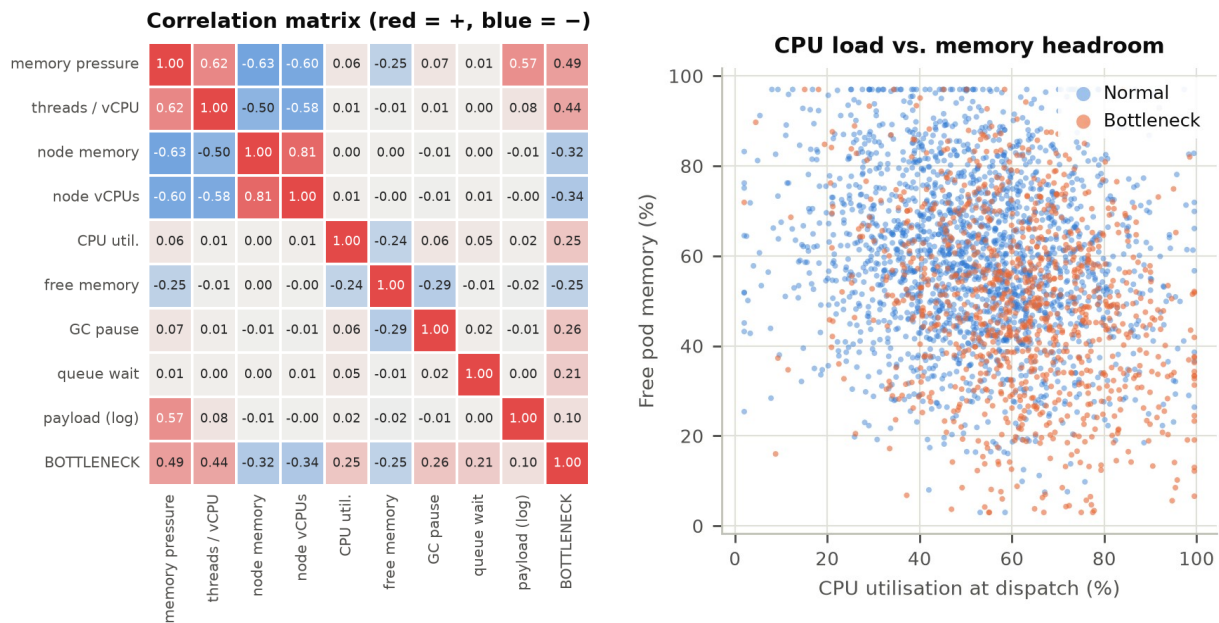


Figure 7. (a) Correlation matrix of key features; (b) CPU utilisation versus free memory at dispatch (3,000 sampled jobs).

The strongest correlations with the target (Figure 7a) are memory pressure ($r = 0.49$), threads per vCPU (0.44), node vCPUs (-0.34) and node memory (-0.32); node memory and vCPUs are collinear (0.81), as are memory pressure and payload (0.57). Bottleneck jobs cluster where CPU is high and memory headroom is low (Figure 7b).

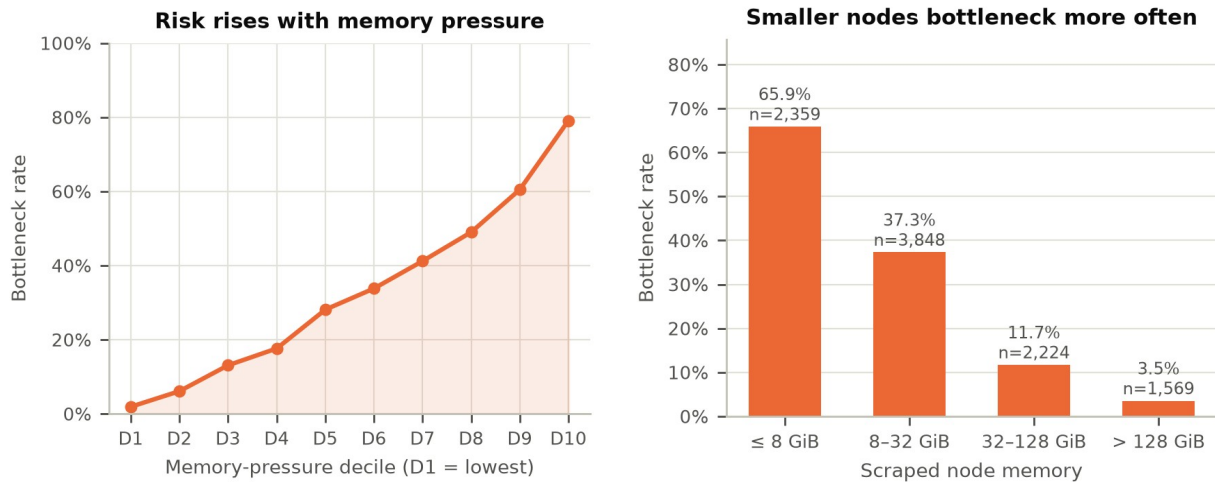


Figure 8. (a) Bottleneck rate across memory-pressure deciles; (b) bottleneck rate by scraped node memory.

Risk climbs steadily from 1.9% in the lowest memory-pressure decile to 79.1% in the highest, and 65.9% of jobs on scraped nodes with ≤ 8 GiB bottleneck against 3.5% on nodes above 128 GiB (Figure 8).

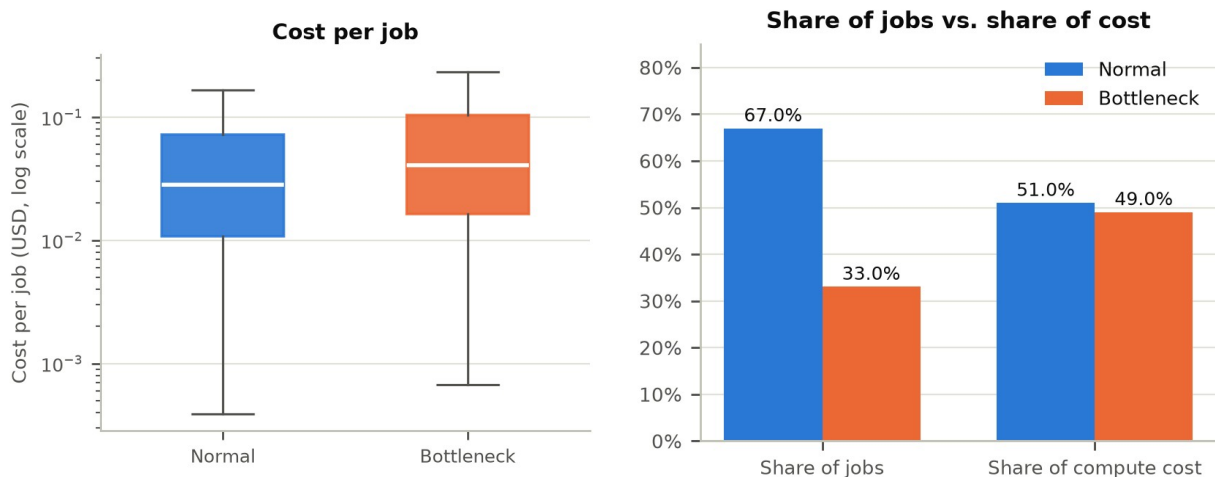


Figure 9. (a) Cost per job by class (log scale); (b) share of jobs versus share of total compute cost.

Bottlenecks create the cost spikes. They are 33.0% of jobs but **49.0% of compute cost** (\$407.56 of \$831.43) (Figure 9). Their median cost is only 1.44× that of normal jobs despite running 6.1× longer, because they run on cheap, undersized nodes (median \$0.174/h versus \$0.680/h): choosing the cheapest node for a heavy job is a false economy.

4. Analytics Method and Implementation

The task is **binary supervised classification** (*Bottleneck* = 1, *Normal* = 0). Three syllabus techniques [7] were compared: **Logistic Regression (LR)**, which models the log-odds as $P(\text{bottleneck} | \mathbf{x}) = 1 / (1 + e^{-(\beta_0 + \sum \beta_i x_i)})$, gives probabilities for a business-driven threshold and interpretable odds ratios; **Decision Tree (DT)**, which yields if-then dispatch rules from Gini-based splits; and **Random Forest (RF)**, a bootstrap ensemble of trees that captures non-linear effects and tests whether extra complexity adds value. Interpretability was required, since engineers will only adopt a dispatch policy they understand.

Implementation (scikit-learn [8] Pipelines, so preprocessing is fitted on training data only):

- **Features:** 22 numeric and 4 categorical pre-execution features (job profile, scraped node specification and price, node state, engineered features). Identifiers and the post-execution outcomes `execution_time_sec` and `job_cost_usd` were excluded to prevent target leakage.

- **Split and encoding:** stratified 75/25 split (7,500 training, 2,500 test jobs); one-hot encoding; standardisation for LR, raw units for the scale-invariant trees (readable split rules); `class_weight = "balanced"`, since a missed bottleneck costs more than a false alarm.
- **Tuning and selection:** 5-fold stratified cross-validated grid search on ROC-AUC – DT: depth {4, 6, 8, 10} × minimum leaf {20, 50, 100} → depth 8, leaf 50; RF (300 trees): depth {10, 16, None} × leaf {2, 5, 10} → depth None, leaf 2. The model with the best CV ROC-AUC was selected and the test set was used once.
- **Metrics and threshold:** accuracy, precision, recall, F1 and ROC-AUC; recall is the primary business metric, and the decision threshold was set on the precision–recall curve to reach ≥ 90 % recall.

5. Comparison with State-of-the-Art Methods

Table 3. Comparison of this case study with recent published studies.

Published study / year	Dataset	Method used	Evaluation metric	Key result	Comparison with this work
Gao, Wang and Shen [2], IEEE Big Data 2019	Google cluster trace (task and job logs)	Multi-layer bidirectional LSTM with priority, resubmission and scheduling-delay inputs	Accuracy	93 % (task) and 87 % (job failure); beat earlier ML baselines	Same goal, but needs the time-series history of a running task; this study uses only dispatch-time data and adds real prices for a cost view.
Jassas and Mahmoud [3], Sensors 2022	Google cluster, LANL Mustang and Trinity traces	DT, RF, KNN, QDA, GB, XGBoost, Naive Bayes with RFE / SelectKBest	Accuracy, precision, recall, F1	RF + RFE ≈ 99 % on Google trace; failures linked to requested CPU/RAM and priority	Confirms resource requests and priority as drivers. Their features include job/machine IDs; identifiers were excluded here to avoid memorisation.
Tengku Asmawi, Ismail and Shen [4], J. Cloud Computing 2022	Google Cluster Traces 2011	LR, DT, RF, GB, XGBoost vs. 1–3-layer LSTM	Accuracy, precision, sensitivity, specificity, F-score	XGBoost best for jobs (F 0.931); DT/RF best for tasks (89.75 %); traditional ML ≥ deep learning	Closest method (LR/DT/RF); both find simple models sufficient. Here LR ranks first as the risk is near log-linear; a recall-oriented threshold is added.
Saxena, Kumar, Singh and Schmid [5], IEEE TPDS 2023	Google cluster CPU/memory; PlanetLab CPU traces	Benchmark of five workload-predictor classes (deep, hybrid, ensemble, quantum NN, ...)	MSE, MAE, RMSE	Quantum NN lowest error, ensembles next; error grows with horizon	Forecasts total demand (regression) for autoscaling; this study classifies single jobs – forecasting sizes the cluster, classification places the job.

Discussion. The published studies use large real traces from Google and national laboratories, whereas this study combines real scraped infrastructure data with simulated telemetry. Because datasets, label definitions and settings differ, **the reported scores are not directly comparable and no method is claimed superior on the basis of its score.** At the level of approach, all studies agree that resource-related attributes (requested CPU/memory, priority, contention) are the strongest predictors—consistent with the memory-pressure and CPU findings here—and [4] also found traditional classifiers competitive with deep networks. This study’s distinct contributions are its strict use of *dispatch-time* information, the link to *real scraped prices* that turns a reliability model into a FinOps tool, and a business-driven operating threshold; its main limitation is the simulated telemetry.

6. Results, Business Insights and Recommendations

Table 4. Five-fold cross-validation and hold-out test performance (threshold 0.5).

Model	CV ROC-AUC (mean ± SD)	Test accuracy	Precision	Recall	F1-score	Test ROC-AUC
Logistic Regression	0.940 ± 0.003	0.859	0.745	0.873	0.804	0.947
Random Forest	0.915 ± 0.006	0.845	0.735	0.832	0.780	0.927
Decision Tree	0.867 ± 0.009	0.774	0.619	0.821	0.706	0.872

Logistic Regression is the best model on every metric (Table 4): CV ROC-AUC 0.940 ± 0.003 and, on the test set, ROC-AUC 0.947, accuracy 85.9%, precision 74.5% and recall 87.3%. Random Forest is second (0.927) and the single tree last (0.872). Test scores match the cross-validation scores, so there is no overfitting; LR wins because the risk relationship is largely monotonic in the log-scaled memory pressure, and extra flexibility adds variance without signal.

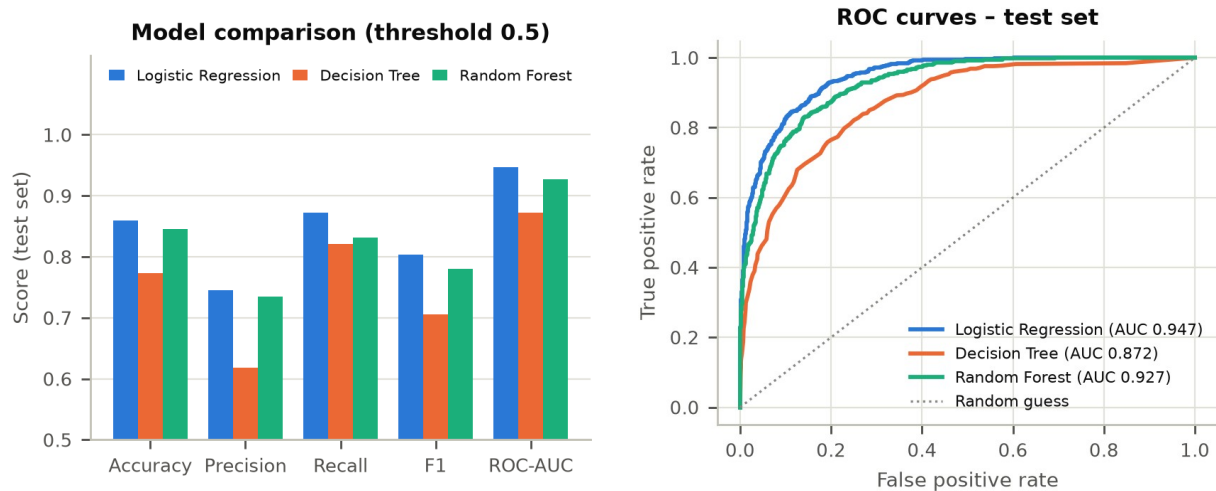


Figure 10. (a) Test-set metrics of the three classifiers; (b) ROC curves.

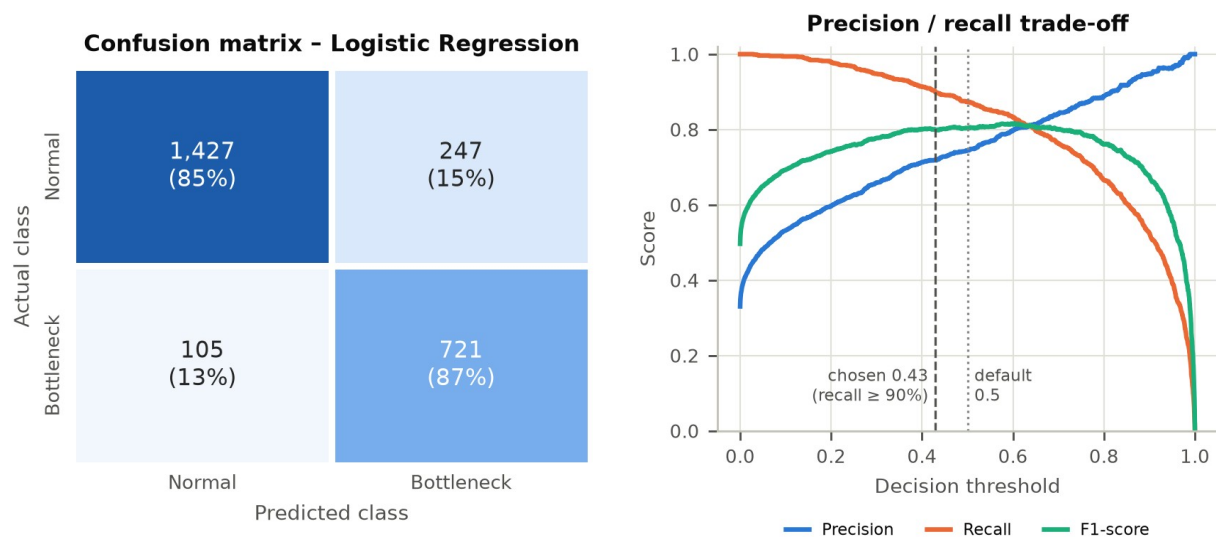


Figure 11. (a) Confusion matrix of Logistic Regression at threshold 0.5; (b) precision, recall and F1 across thresholds (chosen 0.43).

At threshold 0.5 the model catches **721 of 826 bottleneck jobs**; 247 of 1,674 normal jobs are false alarms, which would only be sent to a larger node (Figure 11a). Lowering the threshold to **0.43** raises recall to **90.1%** at 71.9% precision, flagging 41.4% of jobs; the flagged bottleneck jobs account for **83.4% of bottleneck-related spend** (Figure 11b).

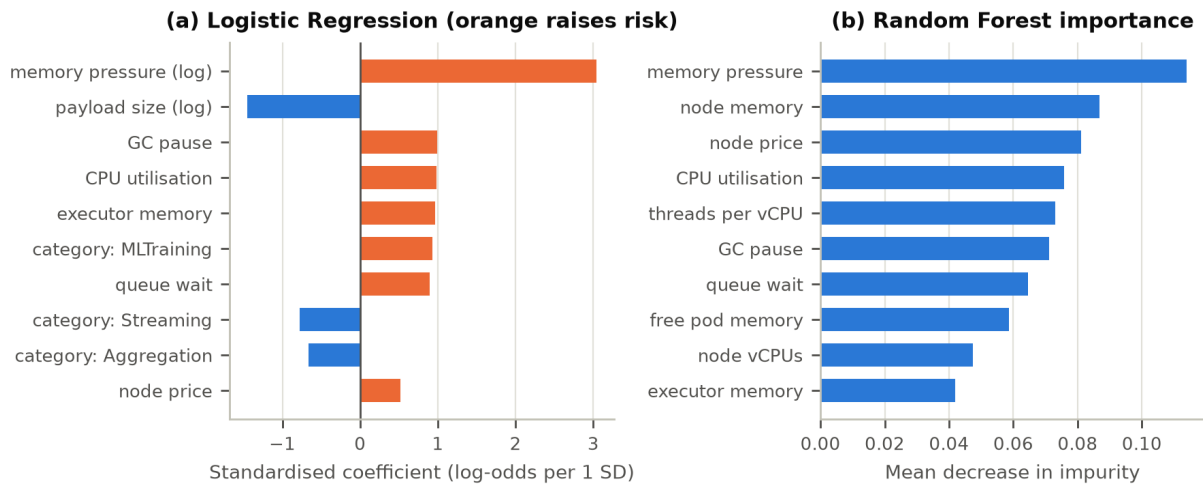


Figure 12. (a) Standardised Logistic Regression coefficients (orange raises risk, blue lowers it); (b) Random Forest importances.

Both models agree on the drivers (Figure 12). In LR a one-standard-deviation rise in memory pressure multiplies the odds of a bottleneck by about **21**; GC pause, CPU load, executor memory, ML-training jobs and queue wait each multiply them by 2.5–2.7, while streaming, aggregation and *Critical* priority lower the risk. The negative payload coefficient reflects its collinearity with memory pressure ($r = 0.57$): payload is risky only relative to free memory. The tuned tree (Figure 13) turns this into rules—first **node memory** ≤ 27 GiB, then CPU above $\approx 52\%$ on small nodes, or memory pressure and GC pause on large ones.

Decision Tree - top three levels (orange = mostly bottleneck, blue = mostly normal)

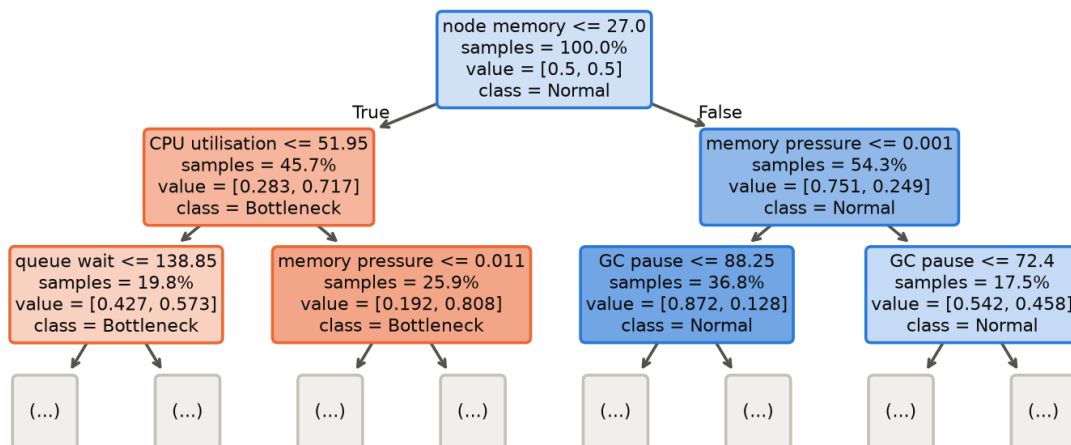


Figure 13. Top three levels of the tuned Decision Tree (class-weighted proportions, raw units).

Key business insights and recommendations

Table 5. Insight-driven recommendations.

Insight (evidence)	Recommendation	Expected impact
Memory pressure drives risk (4% → 70% from lowest to highest deciles)	Compute payload ÷ free memory at dispatch; route high-ratio jobs to memory-optimised nodes	Removes the largest source of bottlenecks
Busy nodes multiply risk (53% at CPU $\geq 75\%$ vs 18% below 40%)	Admission rule: queue or reroute when node CPU $\geq 75\%$	Fewer slow-downs at peak hours
Accurate early warning (recall 90% at threshold 0.43)	Deploy LR as a pre-dispatch check; flagged jobs get a larger node or extra replica	$\approx 83\%$ of bottleneck spend addressed in advance
Risk concentrated in ML training (44%) and web scraping (35%)	Dedicated larger pool for risky categories; pack streaming/aggregation on small nodes	Buffer capacity paid for only where needed
Small nodes are a false economy (49% of cost from 33% of jobs)	Reserve headroom only for the $\approx 41\%$ flagged jobs, not the whole cluster	Cloud spend tied to actual risk (FinOps)
Scraped spot prices $\approx 67\%$ below on-demand	Run low-priority, delay-tolerant jobs on spot capacity	Savings on the 31% of spend from low-priority jobs

7. Conclusion

The study applied the full business-analytics workflow to resource bottlenecks and cost spikes in distributed job processing. 1,413 real cloud instance types were web-scraped and 10,000 job records were generated on top of them, cleaned, explored and modelled. **Objective 1** was met by identifying memory pressure, CPU load at dispatch, thread contention, GC pause and queue wait as root causes; **Objective 2** by a Logistic Regression early-warning model (test ROC-AUC 0.947, 90% recall at the chosen threshold) that uses dispatch-time data only; and **Objective 3** by showing that bottlenecks cause 49.0% of spend and deriving six cost-aware recommendations. **Limitations:** the job telemetry is simulated (node specifications and prices are real), so thresholds such as 27 GiB or 75 % CPU must be re-estimated on a company's own logs; the label follows a resource-pressure mechanism, whereas real systems also fail for hardware or network reasons; and prices were scraped for one region at one time. **Future work:** retrain on real telemetry (e.g. public Google or Alibaba traces), add reserved and multi-region pricing, and validate the dispatch policy in a live A/B test.

References

- [1] Vantage, “EC2Instances.info – Easy Amazon EC2 Instance Comparison,” instances.vantage.sh. [Online]. Available: <https://instances.vantage.sh/> (accessed Sep. 23, 2026).
- [2] J. Gao, H. Wang, and H. Shen, “Task failure prediction in cloud data centers using deep learning,” in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2019. Extended version: *IEEE Trans. Services Computing*, doi: 10.1109/TSC.2020.2993728.
- [3] M. S. Jassas and Q. H. Mahmoud, “Analysis of job failure and prediction model for cloud computing using machine learning,” *Sensors*, vol. 22, no. 5, p. 2035, 2022, doi: 10.3390/s22052035.
- [4] T. N. Tengku Asmawi, A. Ismail, and J. Shen, “Cloud failure prediction based on traditional machine learning and deep learning,” *Journal of Cloud Computing*, vol. 11, 2022, doi: 10.1186/s13677-022-00327-0.
- [5] D. Saxena, J. Kumar, A. K. Singh, and S. Schmid, “Performance analysis of machine learning centered workload prediction models for cloud,” *IEEE Trans. Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1313–1330, 2023, doi: 10.1109/TPDS.2023.3240567.
- [6] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. 26th ACM Symp. Operating Systems Principles (SOSP)*, 2017, pp. 153–167.
- [7] G. Shmueli, P. C. Bruce, P. Gedeck, and N. R. Patel, *Data Mining for Business Analytics: Concepts, Techniques and Applications in Python*. Hoboken, NJ, USA: Wiley, 2020.
- [8] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.
- [10] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [11] L. Richardson, “Beautiful Soup documentation.” [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- [12] Apify, “Web Scraper (apify/web-scraper).” [Online]. Available: <https://apify.com/apify/web-scraper>